

Modül 02 — Yaratımsal Kalıplar I: Fabrikalar

İçindekiler

1	Öğrenme çıktıları	2
2	Motivasyon	2
3	Singleton	2
3.1	Problem	2
3.2	Amaç	2
3.3	Yapı	3
3.4	Klasik Java	3
3.5	Modern Java 27	4
3.6	Singleton neden çoğu zaman bir anti-kalıptır	4
3.7	Ara not: Lazy Constants (önizleme)	4
3.8	Gerçek dünyada kullanımı	5
3.9	Tuzaklar ve ne zaman KULLANILMAMALI	5
3.10	İlgili kalıplar	5
4	Static Factory Method	5
4.1	Problem	5
4.2	Amaç	5
4.3	Yapı	5
4.4	Klasik Java	6
4.5	Modern Java 27	6
4.6	Gerçek dünyada kullanımı	7
4.7	Tuzaklar ve ne zaman KULLANILMAMALI	7
4.8	İlgili kalıplar	7
5	Factory Method	7
5.1	Problem	7
5.2	Amaç	7
5.3	Yapı	8
5.4	Klasik Java	8
5.5	Modern Java 27	9
5.6	İkinci örnek: lojistik	9
5.7	Gerçek dünyada kullanımı	9
5.8	Tuzaklar ve ne zaman KULLANILMAMALI	9
5.9	İlgili kalıplar	9
6	Abstract Factory	9
6.1	Problem	9
6.2	Amaç	9
6.3	Yapı	10
6.4	Klasik Java	10
6.5	Modern Java 27	10
6.6	İkinci örnek: bulut sağlayıcılar	11
6.7	Gerçek dünyada kullanımı	11
6.8	Tuzaklar ve ne zaman KULLANILMAMALI	11
6.9	İlgili kalıplar	11

7	ServiceLoader	12
7.1	Problem	12
7.2	Nasıl çalışır	12
7.3	Örnek	12
7.4	Gerçek dünyada kullanımı	13
7.5	Tuzaklar ve ne zaman KULLANILMAMALI	13
8	Yaratımsal bir teknik seçmek	13
9	Özet	14
10	Sınav	14
11	Ödevler	15
12	İleri okuma	15

3. Hafta · Ön koşullar: m01 (OCP, DIP, bileşim kökü) · Tahmini çalışma süresi: 5 saat

Her	örneği	derlemeden	çalıştırın:
<pre>java modules/m02-creational-factories/src/main/java/io/github/aliturgutbozkurt/ patterns/m02/examples/<yol>/<Demo>.java</pre>			
27)			

1 Öğrenme çıktıları

Bu modülün sonunda şunları yapabilirsiniz:

1. İş parçacığı güvenli bir Singleton (Tekil Nesne) (enum, lazy holder) **yazmak** ve global durumun test edilebilirliğe neden zarar verdiğini **açıklamak**.
2. Anlamli adlara sahip, nesne önbellekleyen ve alt tip seçen statik fabrika metotları **yazmak**.
3. Factory Method (Fabrika Metodu) kalıbını klasik (alt sınıf kancası) ve modern (Supplier, enum kaydı) biçimleriyle **uygulamak**.
4. Bir ürün ailesini tutarlı tutan bir Abstract Factory (Soyut Fabrika) **uygulamak**.
5. ServiceLoader ile uygulamaları çalışma zamanında **yüklemek**.
6. Bir soruna hangi yaratımsal tekniğin uyduğuna **karar vermek** — "doğrudan kurucuyu çağır" dahil.

2 Motivasyon

`new HtmlFormatter()` zararsız görünür. Ama `new` diyen satır *hangi sınıfın* kullanılacağına karar verir; m01 de bize bu tür kararların olabildiğince az yerde durması gerektiğini öğretti (OCP, DIP). Bu modüldeki beş teknik, bu kararı daha iyi bir yere taşır: adı olan bir metoda, bir alt sınıfa, bir aile nesnesine ya da bir yapılandırma dosyasına.

3 Singleton

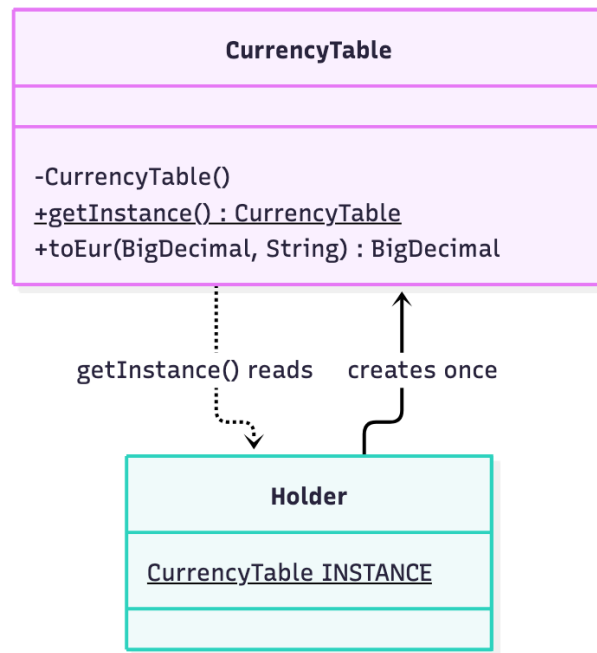
3.1 Problem

Bazı şeyler uygulama başına bir kez var olmalıdır: bir ayarlar nesnesi, yavaş bir servisten yüklenen döviz kurları tablosu. Onları iki kez oluşturmak zaman kaybettirir ya da daha kötüsü, iki kopyanın birbiriyle çelişmesine yol açar.

3.2 Amaç

Bir sınıfın yalnızca **tek bir nesnesi** olmasını sağlamak ve ona global bir erişim noktası sunmak.

3.3 Yapı



3.4 Klasik Java

Private bir kurucu başkalarının `new` demesini engeller; statik bir alan tek nesneyi tutar:

```
// file: examples/singleton/testability/before/SequenceGenerator.java
public final class SequenceGenerator {

    private static final SequenceGenerator INSTANCE = new SequenceGenerator();

    // Mutable state behind a global access point – the problem this example
    // demonstrates.
    private int next = 1;

    private SequenceGenerator() {}

    public static SequenceGenerator getInstance() {
        return INSTANCE;
    }
}
```

Bu nesne, sınıf başlatıldığında oluşturulur. Oluşturmak pahalıysa, **lazy holder** (tembel tutucu) yöntemi bunu ilk `getInstance()` çağrısına kadar erteler. JVM iç içe `Holder` sınıfını yalnızca ilk kullanıldığında başlatır ve sınıf başlatma, dil spesifikasyonu gereği iş parçacığı güvenlidir — `synchronized` yok, çift kontrollü kilitleme yok:

```
// file: examples/singleton/holder/CurrencyTable.java
private static final class Holder {
    static final CurrencyTable INSTANCE = new CurrencyTable();
}

public static CurrencyTable getInstance() {
    return Holder.INSTANCE;
}
```

Testler iki iddiayı da kanıtlar: yeni bir sınıf yükleyicide (class loader) `creations()`, `getInstance()` çalışana kadar 0'dır; aynı anda `getInstance()` çağıran 1000 sanal iş parçacığı, bir kez oluşturulmuş tek bir nesne görür.

3.5 Modern Java 27

Tek elemanlı bir `enum`, en basit doğru Singleton'dır. JVM tek nesneyi, iş parçacığı güvenli oluşturmayı ve ne serileştirmenin ne de reflection'ın ikinci bir nesne üretemeyeceğini garanti eder (ikisi de `EnumSingletonTest` 'te test edilir):

```
// file: examples/singleton/enumsingleton/AppSettings.java
public enum AppSettings {
    INSTANCE;

    private final Map<String, String> values = Map.of(
        "app.name", "PatternShop",
        "currency", "EUR",
        "page.size", "20");
```

3.6 Singleton neden çoğu zaman bir anti-kalıptır

`OrderService` 'in neye bağımlı olduğuna bakın:

```
// file: examples/singleton/testability/before/OrderService.java
public String placeOrder() {
    return "ORD-" + SequenceGenerator.getInstance().next();
}
```

Kurucusunda ya da imzasında hiçbir şey bu bağımlılığı göstermez ve sayaç **global değiştirilebilir durumdur**: iki "bağımsız" servis birbirinin numaralarını sürdürür; bir testin sonucu, ondan önce hangi testlerin çalıştığına bağlı olur. Çözüm m01'deki Bağımlılığın Tersine Çevrilmesi'dir — bağımlılığı isteyin:

```
// file: examples/singleton/testability/after/OrderService.java
public OrderService(IdSource ids) {
    this.ids = Objects.requireNonNull(ids, "ids");
}
```

"Yalnızca tek nesne" artık sınıfın bir özelliği değil, bileşim kökünün verdiği bir karardır:

```
== before: OrderService calls SequenceGenerator.getInstance() ==
a second OrderService continues the first one's numbers: true
== after: the IdSource is injected ==
shared on purpose: ORD-1, ORD-2
separate sources: ORD-1, ORD-1
fixed for a test: ORD-42
```

3.7 Ara not: Lazy Constants (önizleme)

⚠ **Önizleme özelliği — notlandırılan kodda kullanılmaz.** JDK 27'de *Lazy Constants* (JEP 531, üçüncü önizleme) vardır: ilk erişimde bir kez hesaplanan ve sonra JVM tarafından sabit gibi ele alınan bir değeri tanımlamanın JDK destekli yolu. Elle yazılan lazy holder'ların yerini almayı amaçlar. Önizleme API'si olduğu için hâlâ değişebilir; bu ders holder yöntemini kullanır. Ayrıntılar: [JEP 531](https://openjdk.org/jeps/531) ve [docs/java27-features.md](https://docs.java27-features.md).

3.8 Gerçek dünyada kullanımı

`Runtime.getRuntime()` klasik bir Singleton'dır. `Collections.emptyList()` paylaşılan tek bir değişmez nesne döndürür. Spring gibi çatılar "singleton kapsamlı" bean'ler oluşturur — uygulama başına bir nesne, ama global bir erişim noktasından alınmaz, enjekte edilir.

3.9 Tuzaklar ve ne zaman KULLANILMAMALI

- Global değiştirilebilir durum: gizli bağlaşım ve sıraya bağlı testler (yukarıya bakın). Enjeksiyonu tercih edin.
- "Tek nesne" aslında "sınıf yükleyici başına tek nesne"dir — tembellik testi tam da bunu kullanarak yeni bir kopya elde eder.
- Çift kontrollü kilitlemeyle elle yazılan tembel başlatmayı yanlış yapmak kolaydır (`volatile` gerektirir); holder yöntemini ya da bir `enum` kullanın.

3.10 İlgili kalıplar

`getInstance()` metodu bir **Static Factory Method (Statik Fabrika Metodu)**'dur. Soyut fabrikalar çoğu zaman tek nesnedir. m03, genel çözüm olarak DI'yi gösterir.

4 Static Factory Method

4.1 Problem

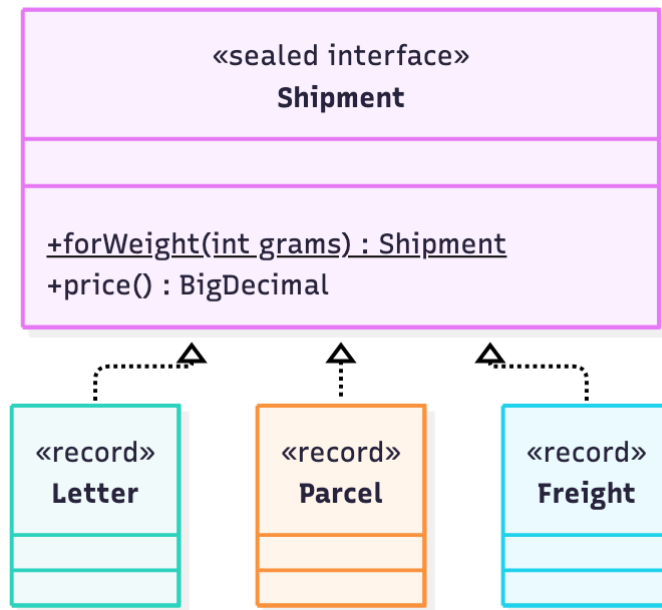
Kurucuların hepsi sınıfın adını taşır, var olan bir nesneyi döndüremez ve her zaman tam olarak kendi sınıflarını döndürür. Bu üç sınır çabuk kendini gösterir: `new Temperature(100)` — Celsius mı, Fahrenheit mı?

4.2 Amaç

Public bir kurucunun yerine (ya da yanında) bir nesne döndüren **statik bir metot** sunmak.

(Bu, 23 GoF kalıbından biri değildir, ama JDK'daki en yaygın yaratımsal tekniktir.)

4.3 Yapı



4.4 Klasik Java

Adlar. Bir fabrika ne yaptığını söyler:

```
// file: examples/staticfactory/Money.java
/** {@code Money.of("12.50", "EUR")}. */
public static Money of(String amount, String currencyCode) {
    return new Money(new BigDecimal(amount), Currency.getInstance(currencyCode));
}

/** Zero in the given currency. */
public static Money zero(String currencyCode) {
    return of("0", currencyCode);
}
```

Adlar, iki fabrikanın aynı parametre listesini paylaşmasına bile izin verir — aşırı yüklenmiş (overloaded) kurucularla bu imkânsızdır:

```
// file: examples/staticfactory/Temperature.java
public static Temperature ofKelvin(double kelvin) {
    return new Temperature(kelvin);
}

public static Temperature ofCelsius(double celsius) {
    return new Temperature(celsius + ZERO_CELSIUS_IN_KELVIN);
}
```

Önbellekleme. `new` her zaman bir nesne oluşturur; bir fabrika ise `Integer.valueOf`'un yaptığı gibi paylaşılan bir nesne verebilir:

```
// file: examples/staticfactory/Percentage.java
/** The shared instance for {@code 0..100}; equal percentages are the same object.
 */
public static Percentage of(int value) {
    if (value < 0 || value > 100) {
        throw new IllegalArgumentException("percentage must be in 0..100: " +
value);
    }
    return CACHE[value];
}
```

4.5 Modern Java 27

Alt tip seçmek. Sealed bir arayüzdeki fabrika, girdiye uyan record'u döndürür; çağıran yalnızca `Shipment` ile çalışır:

```
// file: examples/staticfactory/Shipment.java
/** Letter up to 500 g, parcel up to 30 kg, freight above. */
static Shipment forWeight(int grams) {
    if (grams <= 0) {
        throw new IllegalArgumentException("weight must be positive: " + grams + "
g");
    }
    if (grams <= LETTER_LIMIT_GRAMS) {
        return new Letter(grams);
    }
}
```

```
    return grams <= PARCEL_LIMIT_GRAMS ? new Parcel(grams) : new Freight(grams);
}
```

```
300 g -> Letter 2.50
2500 g -> Parcel 7.50
45000 g -> Freight 53.50
```

JDK genelinde kullanılan adlandırma gelenekleri:

Ad	Anlamı	JDK örneği
of	bileşenlerden oluştur	List.of, LocalDate.of, Path.of
from	başka bir tipten dönüştür	Instant.from(temporal)
valueOf	of gibi, eski stil, çoğu zaman önbel- lekle	Integer.valueOf, String.valueOf
parse	metinden oku	Integer.parseInt, Duration.parse
getInstance / instance	paylaşılan bir nesne döndürebilir	Currency.getInstance
newX	her zaman yeni bir nesne	Files.newBufferedReader

4.6 Gerçek dünyada kullanımı

List.of(...) eleman sayısına göre farklı gizli sınıflar döndürür; EnumSet.of(...) enum'un büyüklüğüne göre long tabanlı ya da dizi tabanlı bir uygulama seçer; Optional.of, Duration.ofMinutes, Path.of.

4.7 Tuzaklar ve ne zaman KULLANILMAMALI

- Yalnızca private bir kurucusu olan sınıf alt sınıflanamaz (çoğu zaman bu bir özelliktir).
- Fabrikalar Javadoc'ta kuruculardan daha zor fark edilir — adlandırma geleneklerine uyun.
- Yalnızca **değişmez** nesneleri önbelleğe alın; paylaşılan değiştirilebilir bir nesne, kılık değiştirmiş bir Singleton'dır.

4.8 İlgili kalıplar

Singleton'ın getInstance() 'ı; Flyweight (Sinek Siklet) (m05) önbelleklemenin ileri götürülmüş hâlidir; çok parametre için Builder (İnşacı) (m03).

5 Factory Method

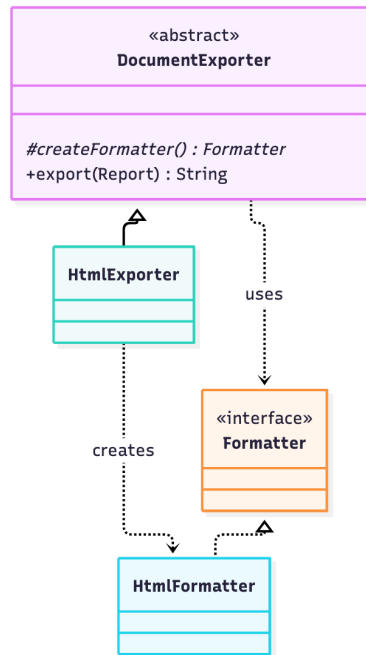
5.1 Problem

Bir dışı aktarıcının tek bir algoritması vardır — başlığı, başlık satırını, her satırı ve sonu yaz — ama ihtiyaç duyduğu biçimlendirici çıktı biçimine göre değişir. Algoritmanın içine bir if (format == ...) koymak OCP'yi ihlal eder.

5.2 Amaç

Nesne oluşturmak için bir arayüz tanımlamak, ama **hangi sınıfın örnekleneceğine alt sınıfların karar vermesini** sağlamak.

5.3 Yapı



5.4 Klasik Java

Oluşturucunun (creator) algoritması bir kez yazılır ve soyut **fabrika metodu**nu çağırır:

```
// file: examples/factorymethod/export/classic/DocumentExporter.java
public abstract class DocumentExporter {

    /** The factory method. */
    protected abstract Formatter createFormatter();

    public final String export(Report report) {
        Formatter formatter = createFormatter();
        var out = new StringBuilder(formatter.begin(report.title()));
        out.append(formatter.header(report.header()));
        for (List<String> row : report.rows()) {
            out.append(formatter.row(row));
        }
        return out.append(formatter.end()).toString();
    }
}
```

Her alt sınıf yalnızca ürünü seçer:

```
// file: examples/factorymethod/export/classic/HtmlExporter.java
public final class HtmlExporter extends DocumentExporter {

    @Override
    protected Formatter createFormatter() {
        return new HtmlFormatter();
    }
}
```

5.5 Modern Java 27

Tek görevi bir kurucuyu çağırmak olan bir alt sınıf fazlasıyla törenseldir. Bir `Supplier<Formatter>` aynı işi görür ve bir enum her biçim için bir kurucu referansı tutabilir:

```
// file: examples/factorymethod/export/modern/ExportFormat.java
public enum ExportFormat {
    MARKDOWN(MarkdownFormatter::new),
    HTML(HtmlFormatter::new),
    CSV(CsvFormatter::new);
}
```

`ExportTest`, klasik ve modern sürümlerin her biçim için birebir aynı metni ürettiğini kanıtlar.

5.6 İkinci örnek: lojistik

`Logistics.planDelivery`, `Transport` 'a göre bir kez yazılmış iş mantığıdır; `RoadLogistics` bir `Truck`, `SeaLogistics` bir `Ship` oluşturur. Bir test, `Logistics` 'e dokunmadan hava yolu seçeneği ekler:

```
// file: examples/factorymethod/logistics/Logistics.java
/** The factory method. */
protected abstract Transport createTransport();

public final String planDelivery(Cargo cargo) {
    Transport transport = createTransport();
}
```

Road: Machine parts by Truck, 1200 km, cost 1440.00, 2 day(s)
Sea: Machine parts by Ship, 1200 km, cost 980.00, 5 day(s)

5.7 Gerçek dünyada kullanımı

`Iterable.iterator()`, JDK'nın en bilinen fabrika metodudur: her koleksiyon kendisine uyan yineleyiciyi oluşturur ve gelişmiş `for` döngüsü yalnızca `Iterator` 'ı bilir. `NumberFormat.getInstance(locale)` ve `Charset.newEncoder()` diğer örneklerdir.

5.8 Tuzaklar ve ne zaman KULLANILMAMALI

- Ürün başına bir alt sınıf patlamaya yol açabilir; alt sınıf başka bir şey yapmıyorsa bir `Supplier` ya da enum kaydı tercih edin.
- Tek bir ürün varsa ve farklılık beklenmiyorsa doğrudan kurucuyu çağırın.

5.9 İlgili kalıplar

`export`, bir adımı fabrika metodu olan bir **Template Method (Şablon Metot)**'tur (m06). Bir **Abstract Factory**, bütün bir aile için fabrika metotları kümesidir.

6 Abstract Factory

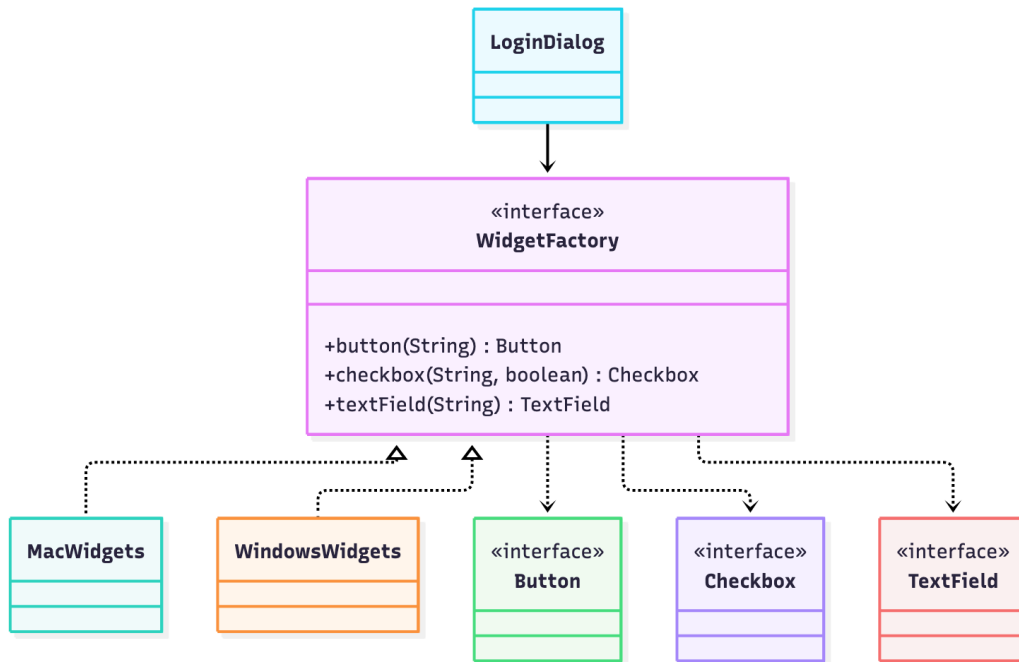
6.1 Problem

Bir giriş penceresi bir metin alanı, bir onay kutusu ve bir düğme ister. macOS'ta hepsi macOS gibi, Windows'ta Windows gibi görünmelidir. Pencere kurucuları doğrudan çağırırsa, unutulmuş tek bir `new` Mac'te bir Windows onay kutusu çıkarır.

6.2 Amaç

Birbiriyle ilişkili **nesne ailelerini**, somut sınıflarını belirtmeden oluşturmak için bir arayüz sunmak.

6.3 Yapı



6.4 Klasik Java

Ailenin her ürünü için bir oluşturma metodu:

```
// file: examples/abstractfactory/ui/WidgetFactory.java
public interface WidgetFactory {

    Button button(String label);

    Checkbox checkbox(String label, boolean checked);

    TextField textField(String label);
}
```

İstemci bütün pencereyi kendisine verilen aileden kurar ve hiçbir somut bileşeni adıyla anmaz:

```
// file: examples/abstractfactory/ui/LoginDialog.java
public LoginDialog(WidgetFactory factory) {
    Objects.requireNonNull(factory, "factory");
    widgets = List.of(
        factory.textField("Username"),
        factory.textField("Password"),
        factory.checkbox("Remember me", true),
        factory.button("Log in"));
}
```

6.5 Modern Java 27

Somut ürünler, fabrikalarının içinde **private iç içe record**'lar olabilir — istemciler onların adını bile yazamaz, dolayısıyla aileleri karıştırmak imkânsızdır:

```
// file: examples/abstractfactory/ui/MacWidgets.java
public final class MacWidgets implements WidgetFactory {
```

```
private static final String PLATFORM = "macOS";

private record MacButton(String label) implements Button {
    @Override public String platform() { return PLATFORM; }
    @Override public String render() { return "(" + label + ")"; }
}
```

Aile, bileşim kökünde, bir statik fabrika tarafından **bir kez** seçilir:

```
// file: examples/abstractfactory/ui/WidgetFactories.java
public static WidgetFactory forOs(String osName) {
    return osName.toLowerCase(Locale.ROOT).startsWith("mac") ? new MacWidgets() :
    new WindowsWidgets();
}
```

```
-- Mac OS X
[macOS] Username: (_____)
[macOS] Password: (_____)
[macOS] ● Remember me
[macOS] ( Log in )
-- Windows 11
[Windows] Username: [_____]
[Windows] Password: [_____]
[Windows] [x] Remember me
[Windows] [ Log in ]
```

6.6 İkinci örnek: bulut sağlayıcılar

İki kurgusal sağlayıcıdan depolama ve mesaj kuyruğu: bir `acme://...` URI'si asla Nimbus kuyruğuna verilmemelidir. `ReportArchiver` iki ürünü de tek bir `CloudFactory`'den alır:

```
// file: examples/abstractfactory/cloud/ReportArchiver.java
public ReportArchiver(CloudFactory cloud) {
    Objects.requireNonNull(cloud, "cloud");
    this.storage = cloud.storage();
    this.queue = cloud.queue();
}
```

6.7 Gerçek dünyada kullanımı

Bir JDBC `Connection`, tek bir veritabanına bağlı bir aile için fabrikadır: `createStatement()`, `prepareStatement(...)`, `createBlob()`. `javax.xml.parsers.DocumentBuilderFactory` ve Swing'in görünüm ve hissi (`UIManager`) diğer örneklerdir.

6.8 Tuzaklar ve ne zaman KULLANILMAMALI

- Yeni bir **ürün** (örneğin bir kaydırıcı) eklemek arayüzü ve her aileyi değiştirir — m01'deki ifade problemi yeniden. Yeni bir **aile** eklemek ise kolaydır.
- Tek bir aile varsa ek arayüzler gereksiz yüküdür.

6.9 İlgili kalıplar

Her oluşturma metodu bir **Factory Method**'dur; fabrika çoğu zaman tek bir nesnedir; **Bridge (Köprü)** (m05) de iki değişim boyutunu birbirinden ayırır.

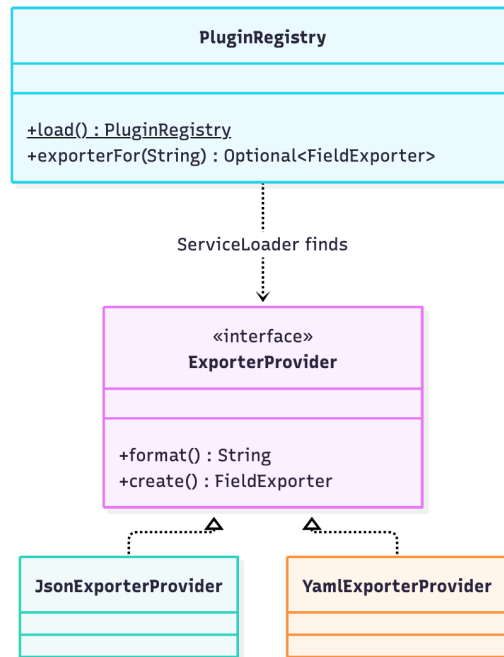
7 ServiceLoader

7.1 Problem

Bir uygulama, kendisi yayımlandıktan *sonra* yazılan dışa aktarma biçimlerini desteklemelidir — ayrı JAR'larda eklentiler olarak. Kod henüz var olmayan sınıfları adıyla anamaz.

7.2 Nasıl çalışır

1. Bir **servis sağlayıcı arayüzü** (service provider interface, SPI) tanımlayın — burada `ExporterProvider`.
2. Her sağlayıcı onu `uygular` ve `public`, argümentsiz bir kurucuya sahiptir.
3. Sağlayıcının JAR'ı onu `META-INF/services/<SPI'nin tam adı>` dosyasında, satır başına bir sınıf adıyla listeler.
4. `ServiceLoader.load(ExporterProvider.class)` onları çalışma zamanında bulur ve örnekler.



7.3 Örnek

Yapılandırma dosyası `src/main/java/META-INF/services/io.github.aliturgutbozkurt.patterns.m02.examples.serviceloader.ExporterPr`

```
# Exporter plugins found by ServiceLoader (see PluginRegistry). One fully qualified
class name per line.
io.github.aliturgutbozkurt.patterns.m02.examples.serviceloader.JsonExporterProvider
io.github.aliturgutbozkurt.patterns.m02.examples.serviceloader.YamlExporterProvider
```

Kayıt (registry) hiçbir sağlayıcı sınıfını adıyla anmaz:

```
// file: examples/serviceloader/PluginRegistry.java
/** A registry over every provider {@link ServiceLoader} finds on the class path. */
public static PluginRegistry load() {
    return new PluginRegistry(ServiceLoader.load(ExporterProvider.class).stream()
        .map(ServiceLoader.Provider::get)
        .toList());
}
```

`ServiceLoader.stream()` tembel `Provider` tutamaçları döndürür: `provider.type()` sınıfı onu örneklemekten **söyler**, `provider.get()` onu oluşturur. Modül sistemiyle aynı kayıt `module-info.java` içinde `provides ... with ...` olarak yazılır.

```
providers on the class path: [JsonExporterProvider, YamlExporterProvider]
formats: [json, yaml]
```

Bu modül yapılandırma dosyasını kaynakların yanında tutar (ve Maven derlemesi için kopyalar); böylece `java PluginDemo.java` eklentileri hiçbir derleme adımı olmadan bulur.

7.4 Gerçek dünyada kullanımı

JDBC sürücüler, `java.nio.file.spi.FileSystemProvider` (zip dosya sistemi), `javax.script` motorları ve `java.time.zone.ZoneRulesProvider`, `ServiceLoader` ile bulunur.

7.5 Tuzaklar ve ne zaman KULLANILMAMALI

- Yapılandırma dosyasındaki bir yazım hatası ancak çalışma zamanında ortaya çıkar (`ServiceConfigurationError`).
- Yineleme sırası belirtilmemiştir — sıra önemliyse, `PluginRegistry`'nin yaptığı gibi sıralayın.
- Birlikte yayımladığınız kod için düz bir fabrika daha basittir ve derleyici tarafından denetlenir.

8 Yaratımsal bir teknik seçmek

Durum	Kullanın
Tek, bariz bir sınıf; farklılık yok	<code>new</code> — gerçekten
Daha açık adlar, önbellekleme, ayırıştırma ya da alt tip seçimi	Static Factory Method
Taban sınıftaki bir algoritma değişen bir ürüne ihtiyaç duyuyor	Factory Method (ya da bir <code>Supplier</code>)
Birbirine uyması gereken birkaç ürün	Abstract Factory
Uygulamalar derleme zamanında bilinmiyor (eklentiler)	<code>ServiceLoader</code>
"Yalnızca bir tane olmalı"	Bileşim kökünün oluşturduğu tek nesne; <code>enum</code> Singleton yalnızca gerçek global sabitler için

9 Özet

Kalıp	Ne zaman kullanılır	Ne zaman kaçınılır	Java 27 kısayolu
Singleton	Tek, durumsuz ya da değişmez global bir kaynak	Bir bağımlılığı gizliyor ya da değiştirilebilir durum tutuyorsa	<code>enum X { INSTANCE }</code>
Static Factory Method	Adlar, önbellekleme ya da alt tip seçimi çağırana yardımcı oluyorsa	Düz bir kurucu yeterince açıksa	Sealed arayüzde fabrika
Factory Method	Taban sınıftaki bir algoritma değişen bir ürüne ihtiyaç duyuyorsa	Alt sınıf yalnızca <code>new</code> diyecekse	<code>Supplier<T>, X::new</code> enum'u
Abstract Factory	Ürünler tek bir aileden gelmeliyse	Tek bir aile varsa	Ürün olarak <code>private</code> iç içe record'lar
ServiceLoader	Yayından sonra eklenen eklentiler	Her şey birlikte yayımlanıyorsa	<code>ServiceLoader.stream()</code>

10 Sınav

1. Neden ne serileştirme ne de reflection bir `enum` Singleton'ın ikinci bir nesnesini oluşturabilir?
2. Lazy holder yönteminin hem tembel hem iş parçacığı güvenli olmasını ne garanti eder?
3. `OrderService` içindeki `SequenceGenerator.getInstance()` 'ın yol açtığı iki somut sorunu sayın.
4. Statik bir fabrika metodunun yapabildiği ama bir kurucunun yapamadığı üç şey söyleyin.
5. `Percentage.of` neden paylaşılan nesneler döndürebilir de değiştirilebilir bir sınıf bunu yapmamalıdır?
6. Klasik dışa aktarıcıda hangi metod fabrika metodu, hangisi şablon metottur?
7. Biçim başına bir alt sınıf yerine ne zaman kurucu referanslarından oluşan bir `enum` tercih edersiniz?
8. Bir Abstract Factory'ye yeni bir *ürün* eklendiğinde ve yeni bir *aile* eklendiğinde neler değişmelidir?
9. `ServiceLoader`, `JsonExporterProvider` 'ı nasıl bulur ve dosyadaki sınıf adı yanlış yazılmışsa ne olur?

Cevaplar

1. Bir `enum` sabiti serileştirmeden geri okunurken var olan sabit adıyla aranır ve `Constructor.newInstance` `enum` nesnesi oluşturmayı reddeder (`IllegalArgumentException`).
2. İç içe `Holder` sınıfı yalnızca `getInstance()` ilk kez `Holder.INSTANCE` 'ı okuduğunda başlatılır ve JVM sınıf başlatmayı bir kilit altında tam olarak bir kez çalıştırır.
3. Bağımlılık gizlidir (kurucuda yoktur) ve global değiştirilebilir sayaç, servislerin ve testlerin birbirini etkilemesine yol açar.
4. Açıklayıcı bir ada sahip olmak, önbellekteki bir nesneyi döndürmek, girdiye göre seçilen bir alt tip döndürmek (ayrıca: karar vermeden önce girdiyi ayrıştırmak).
5. `Percentage` değişmezdir, bu yüzden paylaşım çağırımlara görünmez; değiştirilebilir bir nesneyi paylaşmak değişikliklerin aralarında sızmasına yol açar.
6. `createFormatter()` fabrika metodudur; `export(Report)` onu çağırın şablon metottur.
7. Her alt sınıf tek bir kurucuyu çağırmaktan başka bir şey yapmayacaksa — `enum` biçim başına tek satırdır ve başka sorumluluklar edinemez.
8. Yeni bir ürün, fabrika arayüzünü ve her aileyi değiştirir; yeni bir aile ise yalnızca yeni bir fabrika sınıfıdır.
9. Sınıf yolundaki `META-INF/services/<SPI adı>` dosyasını okur ve listelenen her sınıfı örnekler; yanlış yazılmış bir ad çalışma zamanında `ServiceConfigurationError` fırlatır.

11 Ödevler

- [01 — Static factory'lerle renk değerleri](#) ★★☆
- [02 — Abstract Factory ile oyun seviyeleri](#) ★★☆

12 İleri okuma

- Gamma, Helm, Johnson, Vlissides, *Design Patterns* (1994) — Singleton, Factory Method, Abstract Factory.
- Joshua Bloch, *Effective Java*, 3. baskı (2018), madde 1 (statik fabrika metotları) ve madde 3 (enum singleton).
- Java Dil Spesifikasyonu, [§12.4 Sınıf ve Arayüzlerin Başlatılması](#)
- [java.util.ServiceLoader](#) API belgeleri
- JEP 531 — [Lazy Constants \(Third Preview\)](#)
- Terimlerin Türkçesi: [docs/glossary.md](#)